Looking at the top half, we can infer that 9 must have been inserted before 2 (otherwise, since we assume 9 and 2 would be mapped in the same spot, like in figure C.11, 2 would be stored in A_3). Also, 4 must have been added before 2, because when the algorithm finds out that A_3 is not empty, A_4 would be tried next (since `f(i) = i`, we do a linear scan of the positions following a collision).
Bottom half of the picture: we try to add 7, assuming `h(7) = 3`. Because of open addressing and the definition of function `f`, we try the elements in positions 3, 4 and 5, which are all already used. Then we finally try A_6, which is free, and we add the new key there.

Even with chaining hashing, the number of slots in the table is usually static, as changing the size of the table would change the hash function and therefore the target slot for potentially all elements already stored. This, in turn, would force us to delete every single element from the old table and then insert it in the new one. It's worth noting that this kind of situation often arises with distributed caches (like Cassandra or Memcached) when a new node needs to be added, and - unless proper workarounds have been put in place in a website's architecture - this can cause bottlenecks or even crash a whole site.

C.2.4 Performance

As mentioned, chaining is usually the method of choice because of performance reasons. All operations for a hash table using chaining and linked lists, where the table has size `m` and contains `n` elements, requires on average `O(n/m)` time.

While most of the times it is fine to consider hash tables operations to be `O(1)`, you should keep in mind that the worst case time is `O(n)`. This happens if all elements are mapped to the same element of the table. In such case, the time needed to